Certainly! Here's an overview of the key topics you requested about Dart, covering a range of fundamental to advanced concepts. Each section includes a brief description and relevant examples where applicable:

1. Dart - Comments

- **Description**: Comments are used to explain code and are not executed.

Syntax:
dart
CopyEdit

```
// This is a single line comment

/*
This is a multi-line comment
*/
```

-

2. Dart - Variables

- **Description**: Variables store data values. Dart infers the variable type if not explicitly defined.

Syntax:
dart
CopyEdit

```
var name = 'Alice'; // Automatically inferred as String
int age = 30; // Explicitly typed as Integer
```

-

3. Dart - Operators

- **Description**: Dart supports standard arithmetic, relational, and logical operators.

Example:
dart
CopyEdit

```
var sum = 10 + 5; // Arithmetic operator
var isTrue = sum > 10; // Relational operator
var result = isTrue && false; // Logical operator
```

-

4. Dart - Standard Input Output

- **Description**: Standard input and output operations, typically using `print` for output and `stdin` for input.

Syntax:
dart
CopyEdit
```
import 'dart:io';
print('Enter your name:');
String? name = stdin.readLineSync();
print('Hello, $name!');
```

-

5. Dart - Data Types

- **Description**: The fundamental types in Dart include `int`, `double`, `String`, `bool`, `List`, `Set`, `Map`, etc.

Example:
dart
CopyEdit
```
int age = 30;
double height = 1.75;
String name = "Bob";
bool isValid = true;
```

-

6. Dart - Numbers

- **Description**: Includes `int` and `double` as subtypes.

Example:
dart
CopyEdit
```
int employees = 100;
double price = 99.99;
```

-

7. Dart - Strings

- **Description**: A sequence of characters. Strings can be manipulated and queried.

Example:
dart

CopyEdit

```
String greeting = 'Hello, world';
var substring = greeting.substring(0, 5); // 'Hello'
```

-

8. Dart - List

- **Description**: Ordered collection of items.

Example:
dart
CopyEdit

```
List<int> numbers = [1, 2, 3, 4];
numbers.add(5); // Adds to the end of the list
```

-

9. Dart - Sets

- **Description**: Unordered collection of unique items.

Example:
dart
CopyEdit

```
Set<String> names = {'Alice', 'Bob'};
names.add('Alice'); // Duplicate entries are ignored
```

-

10. Dart - Map

- **Description**: Key-value pairs collection.

Example:
dart
CopyEdit

```
Map<String, int> phoneBook = {'Alice': 12345, 'Bob': 67890};
```

-

11. Queues in Dart

- **Description**: A collection that can be manipulated at both ends. Can be implemented using `List` or special queue classes from `dart:collection`.

Example:
dart

CopyEdit

```
import 'dart:collection';
Queue<int> queue = Queue.from([1, 2, 3]);
queue.addLast(4);
queue.removeFirst(); // 1
```

-

12. Dart - Enums

- **Description**: A way to define named constant values.

Example:
dart
CopyEdit

```
enum Color { red, green, blue }
print(Color.red); // Output: Color.red
```

-

13. Control Flow in Dart

- **Description**: Includes if-else, for loops, while loops, switch-case statements.

Examples:
dart
CopyEdit

```
if (age > 18) {
  print('Adult');
} else {
  print('Minor');
}

for (var i = 0; i < numbers.length; i++) {
  print(numbers[i]);
}
```

-

14. Dart - If-Else Statements

Example:
dart
CopyEdit

```
var age = 20;
if (age >= 18) {
```

```
  print('Adult');
} else {
  print('Minor');
}
```

-

15. Dart - Switch Case Statements

Example:
dart
CopyEdit

```
var grade = 'A';
switch (grade) {
  case 'A':
    print('Excellent');
    break;
  case 'B':
    print('Good');
    break;
  default:
    print('Fail');
}
```

-

16. Dart - Loops

- **Description**: Includes `for`, `while`, and `do-while` loops.

Example:
dart
CopyEdit

```
for (var i = 0; i < 5; i++) {
  print(i);
}

while (isValid) {
  print('Still valid!');
  isValid = false;
}
```

-

17. Dart - Loop Control Statements

- **Description**: `break` and `continue` to control loop execution.

Example:
dart
CopyEdit
```
for (var i = 0; i < 10; i++) {
  if (i == 5) break; // Exits the loop when i is 5
  if (i == 3) continue; // Skips the rest of the loop body when i is
3
  print(i);
}
```

-

18. Labels in Dart

- **Description**: Used with break and continue to control the flow in nested loops.

Example:
dart
CopyEdit
```
outerLoop: for (int i = 0; i < 3; i++) {
  for (int j = 0; j < 3; j++) {
    if (i == j) continue outerLoop;
    print('$i $j');
  }
}
```

-

19. Dart Key Functions

- **Description**: Important built-in functions in Dart, like `print()`, `List.forEach()`, `Map.forEach()`.

Example:
dart
CopyEdit
```
numbers.forEach((number) => print(number));
```

-

20. Dart - Function

- **Description**: A block of code that performs a specific task.

Example:
dart
CopyEdit

```
void greet(String name) {
  print('Hello, $name!');
}
greet('Alice');
```

-

21. Dart - Types of Functions

- **Description**: Includes named functions, anonymous functions, and arrow functions.

Example:
dart
CopyEdit

```
// Named function
void add(int a, int b) {
  print(a + b);
}

// Anonymous function
var multiply = (int a, int b) {
  return a * b;
};

// Arrow function for single expression
int subtract(int a, int b) => a - b;
```

-

22. Dart - Anonymous Function

- **Description**: Functions without a name, often used for passing as arguments.

Example:
dart
CopyEdit

```
var list = ['apples', 'bananas', 'oranges'];
list.forEach((item) {
  print('I love $item');
});
```

-

23. Dart - main() Function

- **Description**: The entry point of a Dart program.

Example:
dart
CopyEdit
```
void main() {
  print('Hello, Dart!');
}
```

-

24. Dart - Common Collection Methods

- **Description**: Methods like `map`, `reduce`, `where`, common to collections like List and Set.

Example:
dart
CopyEdit
```
var numbers = [1, 2, 3, 4, 5];
var evenNumbers = numbers.where((n) => n % 2 == 0);
print(evenNumbers); // [2, 4]
```

-

25. Dart - exit() function

- **Description**: Exits the application.

Example:
dart
CopyEdit
```
import 'dart:io';
exit(0); // Exits the program with exit code 0
```

-

26. Getter and Setter Methods in Dart

- **Description**: Special methods that provide read and write access to an object's properties.

Example:
dart

CopyEdit

```
class Box {
  int _width = 0;

  // Getter
  int get width => _width;

  // Setter
  set width(int val) {
    if (val >= 0) _width = val;
  }
}

var box = Box();
box.width = 10; // Calls the setter
print(box.width); // Calls the getter
```

-

27. Object-Oriented Programming (OOPS) in Dart

- **Description**: Dart supports OOP with classes, objects, inheritance, and polymorphism.

Example:
dart
CopyEdit

```
class Animal {
  void speak() {
    print('The animal makes a sound');
  }
}

class Dog extends Animal {
  @override
  void speak() {
    print('Bark');
  }
}
```

-

28. Dart - Classes & Objects

- **Description**: Basic structure of OOP in Dart.

Example:
dart
CopyEdit
```
class Car {
  String model;
  int year;

  Car(this.model, this.year);

  void display() {
    print('Model: $model, Year: $year');
  }
}

var myCar = Car('Tesla Model S', 2020);
myCar.display();
```

-

29. Dart - Constructors

- **Description**: Special methods that are called when a new object is instantiated.

Example:
dart
CopyEdit
```
class Point {
  double x, y;

  Point(this.x, this.y);
}

var p = Point(1.0, 2.0);
```

-

30. Dart - Super Constructor

- **Description**: Calls the constructor of the superclass.

Example:
dart

CopyEdit

```
class Person {
  String name;
  Person(this.name);
}

class Employee extends Person {
  int employeeID;

  Employee(String name, this.employeeID) : super(name);
}
```

-

31. Dart - this Keyword

- **Description**: Refers to the current instance of the class.

Example:
dart
CopyEdit

```
class Rectangle {
  int width, height;

  Rectangle(int width, int height) {
    this.width = width;
    this.height = height;
  }
}
```

-

32. Dart - static Keyword

- **Description**: Static variables and methods belong to the class rather than any instance.

Example:
dart
CopyEdit

```
class Util {
  static int calculateArea(int width, int height) {
    return width * height;
  }
}
```

```
int area = Util.calculateArea(5, 10);
```

-

33. Dart - super Keyword

- **Description**: Used to refer to immediate parent class object.

Example:
dart
CopyEdit

```
class Parent {
  void msg() {
    print("This is parent class");
  }
}

class Child extends Parent {
  void msg() {
    super.msg();  // Calling the superclass method
    print("This is child class");
  }
}

var obj = Child();
obj.msg();
```

-

34. Dart - Const And Final Keyword

- **Description**: `const` values are compile-time constants. `final` values can only be set once and it is initialized when accessed.

Example:
dart
CopyEdit

```
const double pi = 3.14159;
final DateTime now = DateTime.now();
```

-

35. Dart - Inheritance

- **Description**: Allows a class to inherit properties and methods from another class.

Example:
dart
CopyEdit

```
class Animal {
  void eat() {
    print('The animal is eating');
  }
}

class Dog extends Animal {
  void bark() {
    print('The dog barks');
  }
}
```

-

36. Dart - Methods

- **Description**: Functions defined within classes.

Example:
dart
CopyEdit

```
class Car {
  void startEngine() {
    print('Engine started');
  }
}
```

-

37. Dart - Method Overloading

- **Description**: Dart does not support traditional method overloading where multiple methods have the same name but different parameters. However, you can achieve similar functionality using optional and named parameters.

Example:
dart
CopyEdit

```
void printName(String firstName, [String? lastName]) {
  if (lastName != null) {
    print('$firstName $lastName');
  } else {
```

```
    print(firstName);
  }
}

printName('John', 'Doe');
printName('Jane');
```

-

38. Dart - Getters & Setters

- **Description**: Special methods that provide read and write access to an object's properties.

Example:
dart
CopyEdit

```
class Rectangle {
  double _width = 0;
  double _height = 0;

  // Getter
  double get area => _width * _height;

  // Setter
  set width(double width) {
    if (width >= 0) {
      _width = width;
    }
  }

  set height(double height) {
    if (height >= 0) {
      _height = height;
    }
  }
}

var rect = Rectangle();
rect.width = 10;
rect.height = 5;
print('Area: ${rect.area}');
```

-

39. Dart - Abstract Classes

- **Description**: Classes that cannot be instantiated directly and often contain abstract methods that must be implemented by subclasses.

Example:
dart
CopyEdit

```
abstract class Shape {
  void draw(); // Abstract method
}

class Circle extends Shape {
  @override
  void draw() {
    print('Drawing a circle');
  }
}

var circle = Circle();
circle.draw();
```

-

40. Dart - Builder Class

- **Description**: A design pattern used to construct a complex object step by step. The builder class separates the construction of a complex object from its representation.

Example:
dart
CopyEdit

```
class CarBuilder {
  String? make;
  String? model;
  int? year;

  CarBuilder setMake(String make) {
    this.make = make;
    return this;
  }

  CarBuilder setModel(String model) {
    this.model = model;
```

```
    return this;
  }

  CarBuilder setYear(int year) {
    this.year = year;
    return this;
  }

  Car build() {
    return Car(make: this.make, model: this.model, year: this.year);
  }
}

class Car {
  String? make;
  String? model;
  int? year;

  Car({this.make, this.model, this.year});
}

var car = CarBuilder()
  .setMake('Toyota')
  .setModel('Corolla')
  .setYear(2020)
  .build();
print('Car: ${car.make}, ${car.model}, ${car.year}');
```

-

41. Concept of Callable Classes in Dart

- **Description**: Classes that can be treated as functions.

Example:
dart
CopyEdit

```
class Greeter {
  String call(String name) {
    return 'Hello, $name!';
  }
}
```

```
var greeter = Greeter();
print(greeter('World')); // Calls the Greeter class as if it were a
function.
```

-

42. Dart - Interfaces

- **Description**: Dart does not have a formal interface keyword; instead, classes can act as interfaces and be implemented by other classes.

Example:
dart
CopyEdit

```
class Printer {
  void printData() {
    print('Printing data');
  }
}

class ConsolePrinter implements Printer {
  @override
  void printData() {
    print('Printing to console');
  }
}

var printer = ConsolePrinter();
printer.printData();
```

-

43. Dart - extends Vs with Vs implements

- **Description**: `extends` is used for inheritance, `with` is used for mixing in class functionalities, and `implements` is used for implementing an interface.

Example:
dart
CopyEdit

```
class A {
  void a() => print('A.a()');
}

class B {
```

```
  void b() => print('B.b()');
}

class C extends A with B {
  void c() => print('C.c()');
}

class D implements A, B {
  void a() => print('D.a()');
  void b() => print('D.b()');
  void c() => print('D.c()');
}

var c = C();
c.a(); // Prints 'A.a()'
c.b(); // Prints 'B.b()'
c.c(); // Prints 'C.c()'

var d = D();
d.a(); // Prints 'D.a()'
d.b(); // Prints 'D.b()'
d.c(); // Prints 'D.c()'
```

-

44. Dart Utilities

- **Description**: Dart provides a variety of utilities such as `dart:math` for mathematical functions, `dart:convert` for encoding and decoding JSON, etc.

Example:
dart
CopyEdit

```
import 'dart:math';

var radians = 90 * (pi / 180); // Convert 90 degrees to radians.
var result = sin(radians);
print('The sine of 90 degrees is $result');
```

-

45. Dart - Date and Time

- **Description**: Handling date and time in Dart.

Example:
dart
CopyEdit

```
import 'dart:core';

var now = DateTime.now();
print('Current date and time: $now');

var y2k = DateTime(2000); // January 1, 2000
print('The day of the week on Jan 1, 2000 was ${y2k.weekday}');
```

-

46. Using await async in Dart

- **Description**: Asynchronous programming with futures and async-await.

Example:
dart
CopyEdit

```
Future<void> fetchUserOrder() async {
  // Imagine that this function is fetching user info from another
service or database.
  return Future.delayed(Duration(seconds: 2), () => print('Large
Latte'));
}

void main() async {
  print('Fetching user order...');
  await fetchUserOrder();
  print('Your order is ready.');
}
```

-

47. Data Enumeration in Dart

- **Description**: Iterating over data, typically using `for` or `forEach`.

Example:
dart
CopyEdit

```
var numbers = [1, 2, 3, 4, 5];
numbers.forEach((number) => print(number)); // Prints each number in
the list.
```

-

48. Dart - Type System

- **Description**: Dart is a strongly typed language with both static and dynamic type checking.

Example:
dart
CopyEdit
```
int sum(int a, int b) {
  return a + b;
}

var result = sum(1, 2); // Correct usage
// var wrong = sum('one', 'two'); // This will result in an error.
```

-

49. Generators in Dart

- **Description**: Functions that can produce sequences of values.

Example:
dart
CopyEdit
```
Iterable<int> getRange(int start, int end) sync* {
  for (int i = start; i <= end; i++) {
    yield i; // Yield each number sequentially.
  }
}

print(getRange(1, 5).toList()); // [1, 2, 3, 4, 5]
```

-

50. Dart Programs

- **Description**: A collection of Dart examples to illustrate different concepts.

Example:
dart
CopyEdit
```
void main() {
  print('Hello, Dart!');
}
```

-

51. How to Combine Lists in Dart?

- **Description**: Merging multiple lists into one.

Example:
dart
CopyEdit
```
var list1 = [1, 2, 3];
var list2 = [4, 5, 6];
var combined = [...list1, ...list2]; // Spread operator
print(combined); // [1, 2, 3, 4, 5, 6]
```

-

52. Dart - Finding Minimum and Maximum Value in a List

- **Description**: Calculating the smallest and largest values in a list.

Example:
dart
CopyEdit
```
var numbers = [10, 5, 3, 11, 7];
var max = numbers.reduce((curr, next) => curr > next ? curr : next);
var min = numbers.reduce((curr, next) => curr < next ? curr : next);
print('Max: $max, Min: $min'); // Max: 11, Min: 3
```

-

53. Dart - Splitting of String

- **Description**: Dividing a string into a list of substrings.

Example:
dart
CopyEdit
```
var sentence = 'Hello, world! How are you?';
var words = sentence.split(' ');
print(words); // ['Hello,', 'world!', 'How', 'are', 'you?']
```

-

54. How to Append or Concatenate Strings in Dart?

- **Description**: Combining multiple strings into one.

Example:
dart
CopyEdit

```
var greeting = 'Hello, ';
var name = 'Alice';
var message = greeting + name + '!';
print(message); // 'Hello, Alice!'
```

-

55. How to Find the Length of a String in Dart?

- **Description**: Determining the number of characters in a string.

Example:
dart
CopyEdit

```
var text = 'Hello';
print('The length of the string is ${text.length}'); // 5
```

-

56. Dart - Sort a List

- **Description**: Ordering the elements in a list.

Example:
dart
CopyEdit

```
var numbers = [10, 5, 3, 11, 7];
numbers.sort();
print(numbers); // [3, 5, 7, 10, 11]
```

-

57. How to convert a lowercase string to an uppercase string?

- **Description**: Changing all characters in a string to uppercase.

Example:
dart
CopyEdit

```
var text = 'hello';
var uppercase = text.toUpperCase();
print(uppercase); // 'HELLO'
```

-

58. How to convert all characters of a string to lowercase?

- **Description**: Changing all characters in a string to lowercase.

Example:
dart
CopyEdit
```
var text = 'HELLO';
var lowercase = text.toLowerCase();
print(lowercase); // 'hello'
```

-

59. How to Replace a Substring of a String in Dart?

- **Description**: Modifying part of a string with another string.

Example:
dart
CopyEdit
```
var text = 'Hello, world!';
var replaced = text.replaceAll('world', 'Dart');
print(replaced); // 'Hello, Dart!'
```

-

60. How to Check String is Empty in Dart?

- **Description**: Checking whether a string is empty.

Example:
dart
CopyEdit
```
var text = '';
print(text.isEmpty); // true
```

-

Advance Concepts

61. Exception Handling in Dart

- **Description**: Managing errors in a program.

Example:
dart

CopyEdit

```
try {
  var result = 100 ~/ 0;
} on IntegerDivisionByZeroException {
  print('Cannot divide by zero');
} catch (e) {
  print('Exception occurred: $e');
}
```

-

62. Assert Statements in Dart

- **Description**: Used for debugging to validate assumptions during development.

Example:
dart
CopyEdit

```
var urlString = 'http://example.com';
assert(urlString.startsWith('https'), 'URL ($urlString) should start
with "https".');
```

-

63. Fallthrough Condition in Dart

- **Description**: In switch-case statements, Dart requires explicit fallthrough using `continue`.

Example:
dart
CopyEdit

```
var command = 'OPEN';
switch (command) {
  case 'CLOSED':
    print('The case is closed.');
    break;
  case 'PENDING':
    print('The case is pending.');
    continue open; // Explicit fallthrough
  open:
  case 'OPEN':
    print('The case is open.');
    break;
}
```

-

64. Concept of Isolates in Dart

- **Description**: Dart uses isolates as a way to achieve concurrency. Each isolate has its own memory and runs code in a separate thread.

Example:
dart
CopyEdit

```
import 'dart:isolate';

void printMessage(String message) => print('Message: $message');

void main() async {
  var receivePort = ReceivePort();
  await Isolate.spawn(printMessage, 'Hello from isolate!', onExit:
receivePort.sendPort);
  receivePort.listen((message) {
    print('Received: $message');
    receivePort.close();
  });
}
```

-

65. Dart - Typedef

- **Description**: Typedefs are used to create type aliases in Dart, which can be useful for improving code readability and maintaining a consistent type interface.

Example:
dart
CopyEdit

```
typedef IntList = List<int>;
typedef Compare<T> = int Function(T a, T b);

void sort<T>(List<T> list, Compare<T> compare) {
  // sorting logic
}

int sortInt(int a, int b) => a - b;
var numbers = [3, 2, 1];
sort(numbers, sortInt);
print(numbers); // [1, 2, 3]
```

-

66. Dart - URIs

- **Description**: Handling URIs in Dart to manage data formatted as a Uniform Resource Identifier.

Example:
dart
CopyEdit
```
import 'dart:core';

var uri = Uri.parse('https://example.com/path?name=query');
print('Scheme: ${uri.scheme}');
print('Host: ${uri.host}');
print('Path: ${uri.path}');
print('Query: ${uri.query}');
```

-

67. Dart - Collections

- **Description**: Dart provides a rich set of collection libraries to work with data structures like lists, maps, and sets.

Example:
dart
CopyEdit
```
var list = [1, 2, 3];
var set = {1, 2, 3};
var map = {'first': 1, 'second': 2, 'third': 3};
print(list);
print(set);
print(map);
```

-

68. Dart - Packages

- **Description**: Packages are a way to manage Dart code sharing. They can be created, published, and shared with other developers.

Example:
dart
CopyEdit
```
// Assume you have a package named 'awesome_package' installed
```

```
import 'package:awesome_package/awesome_package.dart';

var awesome = Awesome();
print(awesome.isAwesome);
```

-

69. Dart - Generators

- **Description**: Generators are functions that produce sequences of values lazily. They can be synchronous or asynchronous.

Example:
dart
CopyEdit

```
Iterable<int> getEvenNumbersSync(int start, int end) sync* {
  for (int i = start; i <= end; i++) {
    if (i % 2 == 0) yield i;
  }
}

Stream<int> getEvenNumbersAsync(int start, int end) async* {
  for (int i = start; i <= end; i++) {
    if (i % 2 == 0) yield i;
  }
}

print(getEvenNumbersSync(1, 10).toList()); // [2, 4, 6, 8, 10]
getEvenNumbersAsync(1, 10).forEach(print); // Asynchronously print
2, 4, 6, 8, 10
```

-

70. Dart - Callable Classes

- **Description**: Classes designed to be called like a function.

Example:
dart
CopyEdit

```
class Greeter {
  String call(String name) => 'Hello, $name';
}

var greeter = Greeter();
```

```
print(greeter('World')); // Output: Hello, World
```

-

71. Dart - Isolates

- **Description**: Allows Dart code to run in parallel on separate threads, using message passing to stay safe.

Example:
dart
CopyEdit

```
import 'dart:isolate';

void printMessage(String message) => print(message);

main() async {
  var receivePort = ReceivePort();
  await Isolate.spawn(printMessage, 'Hello from another isolate');
  receivePort.listen((message) {
    print('Received: $message');
    if (receivePort != null) receivePort.close();
  });
}
```

-

72. Dart - Async

- **Description**: Keyword used to define asynchronous functions which can use `await` to pause execution until a Future completes.

Example:
dart
CopyEdit

```
Future<String> fetchUserOrder() async {
  // Simulate a network request delay
  await Future.delayed(Duration(seconds: 2));
  return 'Large Latte';
}

void main() async {
  print('Fetching user order...');
  var order = await fetchUserOrder();
  print('Your order is: $order');
```

```
}
```

-

73. Dart - String codeUnits Property

- **Description**: Accesses the UTF-16 code units of a string.

Example:
dart
CopyEdit
```
var text = 'Dart';
print(text.codeUnits); // [68, 97, 114, 116]
```

-

74. Dart - HTML DOM

- **Description**: Dart can interact with the HTML Document Object Model (DOM) when used in web applications.

Example:
dart
CopyEdit
```
import 'dart:html';

void main() {
  var element = querySelector('#some-id')..text = 'Hello, Dart!';
  document.body!.children.add(element);
}
```

-

These topics cover a comprehensive range of Dart's capabilities, from basic syntax and data structures to advanced concepts like asynchronous programming and isolates. For more detailed information and practical examples, you can refer to the official Dart documentation and other learning resources.